NATURAL LANGUAGE PROCESSING LAB

Experiment 2 : Implement POS Tagging Using spaCy

Implement POS tagging using an existing NLP library such as spaCy and analyze the grammatical category assigned to each token.

Software: Python 3.x, spaCy, Jupyter Notebook / VS Code

1. Background / Theory

Part-of-Speech (POS) tagging is the process of assigning a grammatical category to each token in a sentence. Typical categories include noun, verb, adjective, adverb, pronoun, determiner, preposition, conjunction, auxiliary and punctuation.

1.1 Why POS Tagging is Used

- It provides grammatical information about the words in a sentence.
- It is used as a preprocessing step for many NLP applications.
- It supports syntactic parsing, information extraction, question answering and text analysis.
- It helps distinguish the role of the same word when its grammatical use changes with context.

1.2 POS Tags Used by spaCy

spaCy exposes a coarse-grained Universal Dependencies style POS tag through token.pos_. It also provides a more detailed language-specific tag through token.tag_.

POS	Meaning	Example
NOUN	Noun	student, system
PROPN	Proper noun	India, Python
VERB	Verb	run, analyze

AUX96*	Auxiliary verb	is, have
ADJ	Adjective	good, efficient
ADV	Adverb	quickly, very
PRON	Pronoun	he, they
DET	Determiner	the, a
ADP	Adposition / preposition	in, on
CCONJ	Coordinating conjunction	and, but
SCONJ	Subordinating conjunction	because, although
NUM	Numeral	two, 10
PART	Particle	not, to
INTJ	Interjection	wow, oh
PUNCT	Punctuation	., !, ?

1.3 POS Tag vs. Detailed Tag

The token.pos_ value gives a general grammatical category. The token.tag_ value provides a more detailed tag according to the English language model. For example, a verb may receive a more specific tag indicating tense or grammatical form.

1.4 Context Matters

POS tagging is context-sensitive. A word can receive different tags in different sentences. For example, 'book' is a noun in 'I read a book' but can function as a verb in 'Book a ticket.' This is why a trained NLP model is preferable to simply assigning a fixed tag to every word.

2. Requirements and Setup

- Python 3.x
- spaCy
- English language model: en_core_web_sm
- Jupyter Notebook or VS Code
- Basic knowledge of Python

2.1 Installation

```
# Create and activate a virtual environment if required
python -m venv .venv
```

```
# Windows
.venv\Scripts\activate

# Linux
source .venv/bin/activate

# Install spaCy
python -m pip install --upgrade pip
pip install spacy pandas jupyter

# Download the English model
python -m spacy download en_core_web_sm
```

2.2 Verify the Installation

```
import spacy

print("spaCy version:", spacy.__version__)

nlp = spacy.load("en_core_web_sm")
print("English model loaded successfully.")
```

Expected result: the spaCy version is displayed and the model-loading message is printed.

3. Experiment Workflow

1. Prepare the input text.
2. Load the spaCy English language model.
3. Process the text using the NLP pipeline.
4. Extract each token.
5. Display the coarse POS tag.
6. Display the detailed tag.
7. Display the token's lemma.
8. Display morphological information.
9. Analyze the POS distribution.
10. Test POS tagging on ambiguous words.
11. Record observations and conclusion.

4. Step 1 – Enter the Input Text

I will use the following sample text.

```
text = """
Natural language processing enables computers to analyze human language.
The students are learning NLP, and the researchers are developing efficient
models for processing large collections of documents.
The team will test the system tomorrow.
"""
```

5. Step 2 – Load spaCy and Process the Text

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp(text)

print("Text processed successfully.")
print("Number of tokens:", len(doc))
```

The nlp object contains spaCy's processing pipeline. When the text is passed to nlp(), spaCy tokenizes the text and applies linguistic processing, including POS tagging.

6. Step 3 – Display POS Tags

```
for token in doc:
    print(token.text, "->", token.pos_)
```

Here, token.text is the original token and token.pos_ is its coarse-grained POS category.

6.1 Display POS in a Table

```
import pandas as pd

pos_table = pd.DataFrame({
    "Token": [token.text for token in doc],
    "POS": [token.pos_ for token in doc]
})
```

```
print(pos_table.to_string(index=False))
```

7. Step 4 – Display Detailed POS Tags

```
detailed_table = pd.DataFrame({
    "Token": [token.text for token in doc],
    "POS": [token.pos_ for token in doc],
    "Tag": [token.tag_ for token in doc]
})

print(detailed_table.to_string(index=False))
```

I will compare POS and Tag columns. POS gives a general category, while Tag gives more detailed grammatical information from the English model.

8. Step 5 – Display POS, Lemma and Morphology Together

```
analysis_table = pd.DataFrame({
    "Token": [token.text for token in doc],
    "POS": [token.pos_ for token in doc],
    "Tag": [token.tag_ for token in doc],
    "Lemma": [token.lemma_ for token in doc],
    "Morphology": [str(token.morph) for token in doc]
})

print(analysis_table.to_string(index=False))
```

The lemma gives the base form of a word, while morphological features can describe information such as tense, number, person or degree when the model provides it.

9. Step 6 – Analyze Only Alphabetic Words

```
word_table = pd.DataFrame({
    "Token": [t.text for t in doc if t.is_alpha],
    "POS": [t.pos_ for t in doc if t.is_alpha],
    "Tag": [t.tag_ for t in doc if t.is_alpha],
    "Lemma": [t.lemma_ for t in doc if t.is_alpha]
```

```
})

print(word_table.to_string(index=False))
```

10. Step 7 – Count the Frequency of Each POS Category

```
pos_counts = (
    word_table["POS"]
    .value_counts()
    .rename_axis("POS")
    .reset_index(name="Count")
)

print(pos_counts)
```

I will use this table to identify which grammatical categories occur most frequently in the input text.

10.1 Optional Visualization

```
import matplotlib.pyplot as plt

pos_counts.plot(
    kind="bar",
    x="POS",
    y="Count",
    legend=False,
    title="POS Tag Distribution"
)

plt.xlabel("Part of Speech")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

11. Step 8 – Test Context-Dependent POS Tagging

I will test words whose grammatical category changes according to context. This demonstrates why POS tagging requires contextual analysis.

```
sentences = [
    "I read a book every week.",
    "Please book a ticket for tomorrow.",
    "They watch the match.",
    "They match the colors carefully.",
    "The light is bright.",
    "Please light the lamp."
]

for sentence in sentences:
    print("\nSentence:", sentence)
    sentence_doc = nlp(sentence)

    for token in sentence_doc:
        if token.text.lower() in {"book", "match", "light"}:
            print(
                f"  {token.text:10} POS={token.pos_:8} "
                f"TAG={token.tag_:8} LEMMA={token.lemma_}"
            )
```

I will compare the POS assigned to the same word in different sentences and explain how context changes its role.

12. Step 9 – Create a Final POS Analysis Table

```
final_table = pd.DataFrame([
    {
        "Sentence": token.sent.text,
        "Token": token.text,
        "POS": token.pos_,
        "Tag": token.tag_,
        "Lemma": token.lemma_,
        "Morphology": str(token.morph)
    }
    for token in doc
```

```
])

print(final_table.to_string(index=False))
```

13. Step 10 – Interpret the Results

For my analysis, I will answer the following:

- Which POS category occurs most frequently?
- Which POS categories occur least frequently?
- Which words are tagged as nouns, verbs and adjectives?
- Are auxiliary verbs separated from main verbs?
- Does the same word receive different POS tags in different contexts?
- What additional information is available in token.tag_ compared with token.pos_?
- What morphological information is available for selected words?
- Are there any unexpected POS assignments? If yes, what might have caused them?

14. Expected Observation Format

Token	POS	Detailed Tag	Lemma	Morphology	Observation
students	NOUN	_____	student	_____	Plural noun
are	AUX	_____	be	_____	Auxiliary verb
learning	VERB	_____	learn	_____	Verb form
efficient	ADJ	_____	efficient	_____	Adjective
tomorrow	NOUN	_____	tomorrow	_____	Temporal expression; inspect model tag

15. Result Table – To Be Completed

POS Category	Frequency	Example Tokens	My Interpretation
NOUN	_____	_____	_____
PROPN	_____	_____	_____
VERB	_____	_____	_____
AUX	_____	_____	_____

ADJ	_____	_____	_____
ADV	_____	_____	_____
PRON	_____	_____	_____
DET	_____	_____	_____
ADP	_____	_____	_____
CCONJ	_____	_____	_____
SCONJ	_____	_____	_____
NUM	_____	_____	_____
PART	_____	_____	_____
INTJ	_____	_____	_____

16. Questions

12. What is POS tagging?
13. Why is POS tagging considered a context-dependent NLP task?
14. What is the difference between token.pos_ and token.tag_ in spaCy?
15. What is the difference between a NOUN and a PROPN tag?
16. What is the difference between VERB and AUX?
17. Why can the same word receive different POS tags in different sentences?
18. What information is represented by token.morph?
19. Why do we use a pretrained language model for POS tagging?
20. What are the limitations of automatic POS tagging?
21. How would POS tagging support a downstream NLP application such as named entity recognition or dependency parsing?